

Speaker Script — Word for Word

Duality Gap in Dual Convex Optimization of ReLU Networks

SVC-SPARK Multidisciplinary International Conference 2026

Total talk: 8 minutes. Timing markers are guides, not hard stops.

Slide 1 — Title (0:00–0:15)

(Say nothing. Stand still. Let the title register for 3–4 seconds. Then advance and speak the first line of Slide 2.)

Slide 2 — Motivation: Why Does SGD Work? (0:15–1:30)

Stochastic gradient descent was designed for convex problems. We run it on objectives that are provably non-convex — loss surfaces studded with local minima and saddle points — and despite a decade of optimization theory telling us this should not work, we get models that generalize.

The folk explanation is overparameterization plus good initialization plus some luck. I want to argue today that this explanation is incomplete, and that a much more satisfying answer exists.

The answer is that the problem was secretly convex all along. And the bridge between the non-convex thing you trained and the convex thing you could have solved is called strong duality.

(Advance slide.)

To explain what that means I need to define one object — the duality gap — and then I can state the result that started this programme.

Slide 3 — Duality Gap: Key Definitions (1:30–2:15)

Every optimization problem — what we call the "primal" — has a companion problem, called its "dual," whose optimal value is always a lower bound on the original. This is weak duality, and it is always true, even for non-convex problems.

The gap is the difference between the two: how much the dual underestimates the primal. For convex problems — least squares, Lasso, linear programming — the gap is zero: the dual and primal touch.

For non-convex problems — almost everything in neural networks — the gap is usually strictly positive. The reason this talk exists is that for a specific family of non-convex problems, namely ReLU networks with weight decay, the gap is also zero. That is the result I want to show you first.

(Advance slide.)

That surprising fact is the result from 2020 I want to show you first.

Slide 4 — Convex Reformulation: Pilanci & Ergen (2020) (2:15–4:00)

For a two-layer ReLU network with ordinary weight decay — nothing fancy, no architectural tricks — the non-convex training loss has a finite-dimensional convex reformulation. The two programs have the same optimal value. The duality gap is zero.

Looking at the equation on screen: the first term is the training loss. The second term is weight decay. The sum is over the m hidden neurons. And the bracket-plus notation is the ReLU.

Width m has to be at least some critical value m^* . That m^* is bounded by n plus one — if the network is wider than roughly the number of training points, you are guaranteed zero gap.

The "non-convex" problem you were solving with SGD is, globally, a second-order cone program you could hand to any convex solver. That is a sentence the textbooks do not tell you.

Now — the convex program has order of (n^r) to the power r variables — polynomial in n , exponential in data rank. Clean theoretically, not yet practical. Hold that thought; we will come back to it.

(Advance slide.)

Two layers are secretly convex. What happens when you go deeper?

Slide 5 — Depth vs. Architecture: Effect on the Duality Gap (4:00–5:45)

You might expect the duality gap to grow smoothly with depth — deeper networks are "more non-convex," so the gap should get worse the further you go.

That is not what happens. At L equals two the gap is exactly zero. At L equals three, for a standard deep network with vector output, the gap jumps to something strictly positive. Wang, Ergen, and Pilanci compute it in closed form — it is a gap between a Schatten quasi-norm and the nuclear norm, and it is zero only when all singular values of your label matrix are equal.

(Pause.)

But — and this is the beautiful part — if you rearrange exactly the same network as K independent parallel branches that sum at the output, the gap snaps back to zero at any depth.

By parallel I mean architecturally parallel — not GPU parallelism.

And now look at what modern architectures actually do. Inception, ResNeXt, ResNets with weight decay — these already have parallel branch structure. The paper suggests this may be part of why they train more reliably. Architecture, not depth, is the villain.

(Advance slide.)

So the bridge holds for two-layer networks, and for parallel networks of any depth — but the convex program still has that exponential-in-rank variables problem. Can any of this actually run on real data?

Slide 6 — Polynomial-Time Relaxation: Kim & Pilanci (2024) (5:45–6:30)

Remember the $\binom{n}{r}$ to the r objection I asked you to hold? Here is what happened next.

If you keep only order of $\log-n$ randomly sampled hyperplane arrangement patterns, the resulting small convex program's optimum is within a factor of square-root of $\log-n$ of the true global optimum with high probability. That is the first genuinely polynomial-in-everything algorithm for training regularized ReLU networks with an approximation guarantee.

(Advance slide.)

And in parallel the engineering caught up.

Slide 7 — CRONOS: Scalable GPU Solver — Feng, Frangella & Pilanci (2023) (6:30–7:00)

CRONOS, from Feng, Frangella, and Pilanci in 2023, is a GPU-accelerated ADMM solver that runs convex ReLU training on ImageNet. It matches or beats tuned Adam, with essentially no hyperparameters to sweep.

So the bridge is built, extended, and practical. Let me close with what is still missing.

(Advance slide.)

I want to close with one open question, from a research group outside Stanford, because a survey that only cites one lab is not really a survey.

Slide 8 — Open Question: What Is a "Convex ReLU Network"? (7:00–7:40)

I have been talking about convexifying the training problem. There is a different question — which ReLU networks *represent* a convex function as a map from input to output? This matters when you need the network itself to be convex, as in optimal transport or energy models.

The standard answer has been Input-Convex Neural Networks. In 2025, Gagneux and colleagues showed that from two hidden layers onward, there exist convex functions a ReLU network can express that no ICNN of the same architecture can. The right convex class is still unknown.

This is from a group outside Stanford, which I think is worth noting for a survey talk.

(Advance slide.)

So to summarise where the field stands...

Slide 9 — Takeaway (7:40–8:00)

(Speak slower than normal — the audience is reading the slide as you talk.)

ReLU training was secretly convex. Width gives you the bridge, architecture decides whether it holds at depth, and a square-root of log-n factor makes it tractable in practice.

(Point to the three words at the bottom of the slide.)

Width, architecture, log n. That is the talk.

(Thank the audience. Stop. Hand over to Q&A.)

Backup Slide Scripts (Q&A Only — show only if the trigger question is asked)

B1 — Generalisation, Not Training Loss

Trigger: "Does the zero-duality-gap result say anything about generalization?"

Only training loss. The Pilanci-Ergen programme characterises the global optimum of the regularised training objective — it tells you what the optimal network looks like and how to compute it. It does not on its own give you a bound on test error. There is a natural hope that the polytope structure of the optimal set will eventually feed into a generalisation bound, and there is active work in that direction, but the field has not yet produced a clean theorem of the form "zero duality gap implies generalisation bound X." It is a genuine open direction — arguably the most important one.

B2 — Does This Extend to Transformers?

Trigger: "Does this extend to transformers?"

Not in a published global-optimality sense. The whole convex-reformulation machinery depends structurally on ReLU being piecewise linear, which is what lets you enumerate hyperplane arrangement patterns. Attention is softmax, which is smooth — not piecewise linear — and that structural property does not carry over. Kim and Pilanci explicitly flag transformers as something they want to extend to — that honest flag is itself the answer. It is probably the biggest single open direction in this area.

B3 — Relationship to NTK

Trigger: "How does this relate to the Neural Tangent Kernel?"

NTK is a strict special case of the convex reformulation — this is the result of Dwarknath, Ergen, and Pilanci in 2023. The convex reformulation of a gated ReLU network is an instance of Multiple Kernel Learning, and the NTK is what you get when the mask weights are target-independent — the "lazy" regime where no feature learning happens. Iteratively reweighting the MKL kernel recovers the full convex optimum, and the difference between the NTK and the true convex solution is exactly what we call feature learning. The two views are not in conflict; the convex view contains NTK.

B4 — What About ResNets?

Trigger: "Do skip connections fit into this framework?"

Yes, cleanly. Wang, Ergen, and Pilanci show that a ResNet with weight decay fits the parallel-network framework as a special case — a residual block is structurally a two-branch parallel network, with one branch being the identity transformation. This is actually part of how the paper interprets the empirical success of modern architectures like ResNeXt, Inception, and SqueezeNet: they were already built in a parallel pattern, and that parallel pattern is exactly what restores strong duality at depth beyond two.